

Problem Design

The problems we considered and rejected

15 July 2026

Contents

1 Part 4 — Designing an Original Local Problem	1
1.1 Problem A — Dormitory-Floor Wi-Fi Access-Point Placement (PSO) ★ selected . . .	1
1.2 Problem B — Load-Shedding-Aware Appliance & Study Scheduling in a Dorm Room (GA)	3
1.3 Problem C — Fair Water-Tanker Resupply Routing Across DU Halls (ACO)	4
1.4 4.4 Formal selection of the primary project	5
2 Part B — Designing the Decision-Making Problem	6
2.1 Problem B1 — Overhead Water-Tank Pump Control under Load-Shedding (MDP) ★ selected	7
2.2 Problem B2 — Batch Cooking and Counter Admission in a Hall Dining Room (MDP)	9
2.3 Problem B3 — Shuttle-Bus Dispatch on the Campus Loop (MDP)	10
2.4 4.8 Formal selection of the decision-making project	11

1 Part 4 — Designing an Original Local Problem

Three original optimization problems drawn from the everyday infrastructure of the University of Dhaka (DU) campus and its residential halls. None is a re-skinned textbook benchmark: each is grounded in a concrete, observable local pain point, but is formalised to the standard required for a rigorous swarm / evolutionary treatment. After specifying all three, we formally select one as the primary student project (Part 5 implements it from scratch).

Notation used throughout: bold lowercase = vectors, $\|\cdot\|$ = Euclidean norm, $\mathbb{1}[\cdot]$ = indicator, $\sigma(z) = 1/(1 + e^{-z})$ = logistic function.

1.1 Problem A — Dormitory-Floor Wi-Fi Access-Point Placement (PSO) ★ selected

1.1.1 A.1 Problem statement

Every DU hall resident knows the “dead-zone” problem: a handful of Wi-Fi routers are mounted along a long hall wing, and rooms far from a router — or shadowed by a concrete partition — get unusable signal. Hall administration has a **fixed, tight budget of K access points (APs)** for a floor and must decide *where on the floor plan to mount them* so that the **occupancy-weighted fraction of rooms with a usable link is maximised**, while respecting a co-channel-interference planning rule (APs on the same channel must not sit too close) and the physical boundary of the floor.

Formally: given a floor $[0, W] \times [0, H]$ (metres), N rooms at fixed positions $\mathbf{r}_i$ with occupancy weights w_i , and interior walls that attenuate signal, choose the continuous coordinates of K APs to maximise weighted coverage.

1.1.2 A.2 Why the landscape is a natural, non-trivial fit for swarm intelligence

- **Continuous and moderate-to-high dimensional.** A solution is a point in $\mathbb{R}^{2K}$. PSO operates natively on continuous vectors — no encoding gymnastics.
- **Non-differentiable.** Coverage uses $\max_j \text{RSSI}_{ij}$ (best-AP selection), a soft threshold, and **discrete wall-crossing counts**. The last makes the objective piecewise-defined with kinks: an AP sliding a few centimetres so that a wall enters/leaves its line-of-sight to a room causes a sudden step. No usable gradient exists.
- **Multimodal.** Many spatially different layouts achieve similar coverage (permuting which AP covers which region), so the objective has many local optima and broad symmetric basins — exactly the regime where a *population* that samples the whole floor beats a single hill-climber.
- **Black-box-friendly.** The objective is cheap to *evaluate* but has no closed form to *invert*; we can only sample it. That is the metaheuristic setting.

Gradient descent has no gradient and would need finite-difference surrogates that the wall kinks corrupt; an exact solver (e.g., MILP after discretising the floor) explodes combinatorially and throws away the continuous structure.

1.1.3 A.3 Recommended algorithm and structural justification

Particle Swarm Optimization (PSO), canonical inertia-weight form (Kennedy & Eberhart 1995; inertia by Shi & Eberhart 1998). Each *particle* is *literally a candidate AP layout* — a point in $\mathbb{R}^{2K}$ — so the velocity update $\mathbf{v} \leftarrow w\mathbf{v} + c_1 r_1 (\mathbf{p}_{\text{best}} - \mathbf{x}) + c_2 r_2 (\mathbf{g}_{\text{best}} - \mathbf{x})$ moves layouts through the floor in a way that is geometrically meaningful. The swarm's collective spatial search matches the spatial nature of the problem, and inertia decay gives a clean exploration→exploitation schedule. (A real-valued GA is a viable alternative; PSO is preferred for its native continuous geometry and its uniquely *visualisable* swarm — see the selection in §4.4.)

1.1.4 A.4 Complete solution encoding

A particle is the flat vector

$$\mathbf{x} = [a_0^x, a_0^y, a_1^x, a_1^y, \dots, a_{K-1}^x, a_{K-1}^y] \in \mathbb{R}^{2K},$$

where (a_j^x, a_j^y) are the metre-coordinates of AP j . Bounds: $a_j^x \in [0, W]$, $a_j^y \in [0, H]$. In the reference instance $K = 3$, so the search space is $\mathbb{R}^6$ (a 60 m × 40 m hall block, $N = 40$ rooms).

1.1.5 A.5 Exact fitness function

Signal from AP j to room i uses the **log-distance indoor path-loss model** (2.4 GHz) with discrete wall attenuation:

$$\text{RSSI}_{ij} = P_{\text{tx}} - (L_0 + 10 n \log_{10} \max(d_{ij}, 1)) - \gamma c_{ij}, \quad d_{ij} = \|\mathbf{r}_i - \mathbf{a}_j\|,$$

where P_{tx} = AP transmit power (dBm), L_0 = reference loss at 1 m, n = path-loss exponent, γ = attenuation per wall, and $c_{ij} \in \{0, 1, \dots\}$ is the number of partition walls the segment $\mathbf{r}_i \rightarrow \mathbf{a}_j$ crosses. Each room hears its **best** AP, and coverage is a **soft threshold**:

$$\rho_i(\mathbf{x}) = \sigma\left(\frac{\max_j \text{RSSI}_{ij} - \tau}{s}\right) \in (0, 1),$$

with usable-link threshold τ (dBm) and softness s . The objective maximised is occupancy-weighted coverage minus a separation penalty:

$$F(\mathbf{x}) = 100 \cdot \underbrace{\frac{\sum_i w_i \rho_i(\mathbf{x})}{\sum_i w_i}}_{\text{weighted coverage \%}} - \lambda_{\text{sep}} \sum_{j < k} [\max(0, d_{\min} - \|\mathbf{a}_j - \mathbf{a}_k\|)]^2$$

Symbols: w_i = room occupancy (students), τ, s radio constants, $d_{\min}$ = minimum AP separation (co-channel rule), λ_{sep} = penalty weight.

1.1.6 A.6 Real-world constraints and how they are handled

Constraint	Type	Handling
AP inside floor $[0, W] \times [0, H]$	boundary	Repair : clamp position, zero the offending velocity component (absorbing wall)
Min AP separation $d_{\min}$	inequality	Soft penalty (quadratic breach) inside F
Exactly K APs (budget)	cardinality	Structural (fixed vector dimension $2K$)

1.1.7 A.7 Data source / simulation paradigm

A deterministic, seeded synthetic instance built from realistic DU-hall dimensions: rooms on a jittered grid (real rooms are not perfectly aligned), occupancy weights sampled per room, and interior concrete partitions as line segments. Radio constants are standard 2.4 GHz indoor values. The instance is fully reproducible; it can be swapped for a surveyed floor plan + measured occupancy without changing the optimizer.

1.1.8 A.8 Strong baselines

- **Random Search** at the *identical* fitness-evaluation budget.
- **Grid Search**: exhaustive enumeration of AP combinations over a coarse candidate-site grid, within the same budget.

1.1.9 A.9 Concrete validation methodology

15 independent seeded runs; report mean $\pm$ std of best fitness; **Wilcoxon signed-rank test** (PSO vs Random, paired by seed) for statistical significance; convergence curve (fitness vs iteration) with baseline reference levels; spatial coverage heatmap with the swarm global-best trajectory; swarm-diversity curve as evidence of the exploration→exploitation transition.

1.2 Problem B — Load-Shedding-Aware Appliance & Study Scheduling in a Dorm Room (GA)

1.2.1 B.1 Problem statement

Dhaka load-shedding is a daily reality. A hall room runs off the grid when it is up and off a **shared IPS / mini-inverter with a hard instantaneous power cap and a limited daily energy budget** when it is down. The resident owns a set of M loads/tasks — ceiling fan, tube light, study lamp, laptop charge, phone charge, electric kettle, router — each with a power draw, a duration or duty requirement, a **utility/priority**, and preferred **time windows** (study lamp in the evening, fan overnight). Given the day's known load-shedding schedule, decide **which loads run in which time slots** to maximise total utility without ever exceeding the instantaneous power cap or the daily energy budget.

1.2.2 B.2 Why the landscape fits a Genetic Algorithm

The decision is a **binary schedule matrix** with coupled, non-linear constraints (a knapsack-over-time / constrained scheduling problem). The feasible region is combinatorial and non-convex; utility is separable but the power/energy caps couple all loads within and across slots. This is the textbook home for a **binary GA** — the exact representation in the course slides (binary chromosome, single-point/uniform crossover, bit-flip mutation).

1.2.3 B.3 Recommended algorithm and structural justification

Genetic Algorithm with binary encoding, tournament selection, uniform crossover, and bit-flip mutation (Holland 1975). Bit-flip mutation toggles a single (load, slot) decision — a semantically

meaningful move; crossover mixes two partial schedules. A constraint-repair operator restores feasibility after variation.

1.2.4 B.4 Complete solution encoding

A binary matrix $A \in \{0, 1\}^{M \times T}$ over T time slots (e.g., 48 half-hour slots), $A_{m,t} = 1$ iff load m is energised in slot t ; flattened to a chromosome of length MT .

1.2.5 B.5 Exact fitness function

$$F(A) = \sum_{m=1}^M \sum_{t=1}^T u_m A_{m,t} \alpha_{m,t} - \lambda_P \sum_t [\max(0, \sum_m P_m A_{m,t} - P_{\text{cap}}(t))]^2 - \lambda_E [\max(0, \sum_{m,t} P_m A_{m,t} \Delta t - E_{\text{day}})]^2$$

where u_m = per-slot utility of load m , $\alpha_{m,t} \in \{0, 1\}$ = availability mask (time-window / grid-up indicator), P_m = power draw (W), $P_{\text{cap}}(t)$ = instantaneous cap in slot t (larger when grid is up), Δt = slot length, E_{day} = daily energy budget, λ_P, λ_E = penalty weights.

1.2.6 B.6 Constraints and handling

Instantaneous power ($\sum_m P_m A_{m,t} \leq P_{\text{cap}}(t)$) and daily energy caps as **soft penalties** above; **repair** operator greedily switches off the lowest utility-per-watt load in any over-cap slot; time windows enforced by the mask $\alpha_{m,t}$; minimum contiguous runtime (e.g., kettle) via a penalty on fragmented activations.

1.2.7 B.7 Data source / simulation paradigm

Real appliance wattages + a representative DPDC/DESCO load-shedding block schedule; utilities elicited on a simple priority scale. Fully simulatable.

1.2.8 B.8 Strong baselines

Greedy by utility-per-watt; priority-ordered earliest-feasible scheduling; random search — all at matched evaluation budget.

1.2.9 B.9 Validation methodology

Total utility achieved, feasibility rate, and per-slot load profile (Gantt + power-envelope chart) vs baselines; multi-seed mean $\pm$ std; significance test.

1.3 Problem C — Fair Water-Tanker Resupply Routing Across DU Halls (ACO)

1.3.1 C.1 Problem statement

During a WASA supply disruption, a **water tanker of fixed capacity** must resupply the DU residential halls (e.g., Zia, S.M., F.H., Jagannath, Rokeya, Shamsun Nahar, ...) from a pump station. Each hall has a demand (litres); the tanker makes one or more capacity-limited trips from the station. Route the tanker to **minimise total travel distance/time** (fuel + delay) while meeting demand and per-trip capacity — a **Capacitated Vehicle Routing Problem (CVRP)**.

1.3.2 C.2 Why the landscape fits Ant Colony Optimization

CVRP is a permutation/graph construction problem over hall-to-hall edges — the canonical home of **ACO**, whose artificial ants *build* routes edge-by-edge and lay pheromone on good edges (Dorigo et al. 1996). Shorter, feasible routes get reinforced; evaporation prevents lock-in. The construction-plus-reinforcement mechanic maps one-to-one onto route building.

1.3.3 C.3 Recommended algorithm and structural justification

Ant System / MAX-MIN Ant System: transition probability $p_{xy}^k = \frac{\tau_{xy}^\alpha \eta_{xy}^\beta}{\sum_{z \in \mathcal{N}^k} \tau_{xz}^\alpha \eta_{xz}^\beta}$ with $\eta_{xy} = 1/d_{xy}$; pheromone update $\tau_{xy} \leftarrow (1 - \rho)\tau_{xy} + \sum_k \Delta\tau_{xy}^k$, $\Delta\tau_{xy}^k = Q/L_k$ for edges on ant k 's tour of length L_k .

1.3.4 C.4 Complete solution encoding

An ant's solution is a **giant tour permutation** of halls, split into capacity-feasible sub-routes by a decoder that opens a new trip (returns to the station) whenever the next hall would breach tanker capacity. Pheromone lives on a $(|H| + 1) \times (|H| + 1)$ hall/station edge matrix.

1.3.5 C.5 Exact fitness / objective

$$\min L(\pi) = \sum_{\text{edges } (x,y) \in \text{routes}(\pi)} d_{xy} + \lambda_{\text{tw}} \sum_h \max(0, \text{arr}_h - \text{due}_h)$$

where d_{xy} = road distance, and the second term penalises late arrivals against a due time due_h (optional time windows).

1.3.6 C.6 Constraints and handling

Tanker capacity via **repair in the decoder** (trip closes when full); demand satisfaction is structural (every hall visited once); time windows as **soft penalty**.

1.3.7 C.7 Data source / simulation paradigm

Real DU hall coordinates $\rightarrow$ road distance matrix (OSM); synthetic per-hall demands. Simulatable end-to-end.

1.3.8 C.8 Strong baselines

Nearest-neighbour construction; Clarke-Wright savings heuristic; random feasible routing.

1.3.9 C.9 Validation methodology

Total distance vs baselines; route map overlay; convergence of best-tour length; multi-seed statistics.

1.4 4.4 Formal selection of the primary project

Selected: Problem A — Dormitory-Floor Wi-Fi AP Placement, solved with PSO.

We score all three on the three criteria the assignment names — Feasibility, Novelty, Demonstrability — plus fit to the course's flagship algorithms.

Criterion	A · Wi-Fi (PSO)	B · Scheduling (GA)	C · Water routing (ACO)
Feasibility (codeable from scratch)	★★★ PSO core ≈ 40 lines; objective is pure numpy; no external data	★★ needs constraint-repair + penalty tuning	★ needs distance matrix, trip decoder, pheromone bookkeeping
Novelty (not a textbook benchmark)	★★★ dorm Wi-Fi dead-zones as continuous max-coverage — fresh, relatable	★★ constrained scheduling is common	★ CVRP is a classic textbook problem

Criterion	A · Wi-Fi (PSO)	B · Scheduling (GA)	C · Water routing (ACO)
Demonstrability (visual, viva-impressive)	★★★ particles <i>are</i> 2-D coordinates → live spatial swarm convergence + coverage heatmap	★★ Gantt chart (static)	★★ route map (static)
Course fit	PSO (flagship, continuous)	GA (flagship, binary)	ACO (flagship, discrete)

1.4.1 Why A wins, in detail

- **Feasibility.** PSO from scratch is the smallest, cleanest optimizer of the three: an inertia-weight velocity update and a clamp. The fitness is a handful of vectorised numpy lines. There is no external dataset dependency (unlike C, which wants a real road matrix) and no elaborate repair machinery (unlike B). It is the one we are most confident we can implement correctly and defend line by line.
- **Novelty.** “Where do we bolt the routers so the far rooms stop dropping?” is a problem every examiner has personally suffered, yet it is *not* a canned benchmark (no Rastrigin, no plain TSP). It is a genuine continuous facility-location / max-coverage instance grounded in DU hall life. The academic lineage is real and current (e.g., PSO/Cuckoo base-station siting in 5G — see Part 2), which lends the toy the credibility of a research problem.
- **Demonstrability.** This is the decisive edge. Because a particle **is** an (x, y) layout, the swarm can be drawn *on the floor plan*: one watches the global-best APs migrate across iterations and settle into a spread that lights up a coverage heatmap. That directly satisfies the assignment’s demand for a “spatial visualization of the swarm coordinates ... over time” in a way a Gantt chart or route map cannot match. Combined with a convergence curve overtaking both baselines and a Wilcoxon-significant margin, it makes an unusually legible, persuasive viva artifact.

Part 5 implements Problem A exactly as specified in §A.4–A.9, from scratch, with matched-budget Random and Grid Search baselines and full statistical validation.

2 Part B — Designing the Decision-Making Problem

The swarm half asked *where do we put things*. This half asks *what do we do now, given that we will have to act again in an hour*. Three original sequential decision problems, again drawn from DU campus life, again formalised properly — here as Markov Decision Processes rather than as objective functions. After specifying all three we formally select one, and Part 5 implements it with Value Iteration and Q-learning from scratch.

For each candidate we give: the informal statement; the MDP tuple $(\mathcal{S}, \mathcal{A}, P, R, \gamma)$ with the actual $|\mathcal{S}|$; why the problem is *genuinely* sequential, i.e. an argument that a greedy rule must fail; whether an explicit transition model $P(s' \mid s, a)$ can honestly be written down (this is the question that decides whether Value Iteration is even applicable); baselines; and validation method.

One thing worth stating before the candidates, because it shaped all three: an MDP only rewards lookahead where an action is **irreversible** or a resource is **exhaustible**. If every mistake can be undone next step at a bounded price, the optimal policy collapses towards the greedy one and any planning-versus- learning experiment built on top of it measures nothing. We learned that the hard way; §4.9 is the post-mortem.

2.1 Problem B1 — Overhead Water-Tank Pump Control under Load-Shedding (MDP) ★ selected

2.1.1 B1.1 Informal statement

A DU residential hall stores its water in an overhead tank and refills it with an electric pump. Three things turn this from plumbing into a decision problem. The grid drops out, and it drops out precisely in the evening hours when the hall wants water — and evening outages are *long*: once the power goes it tends to stay gone. A diesel generator can drive the pump through an outage, but the hall buys a **fixed ration of diesel each morning**; burn it at 07:00 on a hunch and there is none left at 21:00 when it actually matters. And demand is not flat — it spikes when the hall bathes, morning and evening.

Every hour the caretaker picks one of three things: leave the pump off, run it on grid power, or burn diesel. Pumping early on cheap grid power banks water against an outage that has not happened yet. Pumping diesel early spends a ration you cannot get back. That is the whole game.

2.1.2 B1.2 The MDP tuple

State $s = (L, t, g, F)$:

Component	Range	Meaning
L	0 ... 10	tank level in 100 L units (1000 L tank)
t	0 ... 23	hour of day — <i>wraps</i> ; this is a continuing task
g	$\{0, 1\}$	is the grid up?
F	0, 1, 2	generator-hours of diesel left today

$$|\mathcal{S}| = 11 \times 24 \times 2 \times 3 = \mathbf{1584}.$$

Actions $\mathcal{A} = \{\text{idle, pump/grid, pump/diesel}\}$, restricted per state: pump/grid requires $g = 1$, pump/diesel requires $F > 0$. That leaves **3432 legal** (s, a) **pairs out of 4752** — 792 states have the grid down and 528 have no diesel left. The masking is not cosmetic; see §B1.5 and Part 5 §5.7 Q5.

Transition $P(s' | s, a)$ factorises into two independent pieces:

- hourly demand $D \sim \text{Poisson}(\lambda_t)$, truncated at 6 and **renormalised** (not clipped — we do not pile the tail onto the top bin), with λ_t following a two-peak daily profile (morning bathing, evening bathing);
- the grid as a two-state Markov chain with hour-dependent $P(\text{fail} | \text{up})$ and $P(\text{restore} | \text{down})$. Evening is the worst of both: failures are likeliest (0.35) *and* restoration is slowest (0.15, so an expected outage of ~6.7 h).

Dynamics: pump first, then the students draw. $L' = \max(0, \min(L + \text{inflow}, 10) - D)$; overflow is charged; the diesel drum is redelivered at midnight and **does not roll over**.

Reward $R(s, a) = -(\text{pump cost} + 0.5 \cdot \text{overflow} + 20 \cdot \mathbb{E}_D[\text{unmet}])$, with pump cost 1.0 on grid and 6.0 on diesel. Note that the realised reward depends on the realised demand D , and D is *not* recoverable from s' (once the tank hits zero you cannot tell how much demand went unmet). So r is not a function of (s, a, s') — this is the disturbance form $r = r(s, a, w)$ with $w = (D, g')$. Value Iteration only ever needs $R(s, a) = \mathbb{E}_w[r]$, which is what we tabulate; the Bellman operator is still a γ -contraction. Q-learning must consume the **realised** r , or the demand distribution would have leaked into the supposedly model-free agent.

Discount $\gamma = 0.99$. One step is one hour, so $1/(1 - \gamma) = 100$ h of effective lookahead — comfortably past the 24-hour cycle the whole story depends on. $\gamma = 0$ is kept as the myopic control.

2.1.3 B1.3 Why it is genuinely sequential

The claim “greedy fails here” is not an assertion we are willing to make for free, so we measure it two ways.

- **The formally correct greedy policy** — value iteration at $\gamma = 0$, i.e. maximise this hour's expected reward and ignore the future entirely — loses **120.62%** of the optimal value. It is not a strawman: it is exactly what “act myopically” means, done correctly, and it is catastrophic. It never pumps pre-emptively, because pumping always costs something *this* hour and the payoff is entirely in the next few.
- **The best reactive rule** — the two-threshold caretaker policy (grid up and tank below $\theta_g \rightarrow$ pump; grid down and tank below $\theta_f \rightarrow$ burn diesel), with **both thresholds tuned by grid search** so the heuristic gets its best shot — still loses **14.51%**. It has the same action set as the optimal policy, including the generator. What it cannot do is read a clock: it does not know that 18:00 is coming.

The mechanism is visible in a single state. At 14:00, tank 4/10, grid up: pumping costs **0.97 more** this hour than idling, and the optimal policy pumps anyway, because its Q^* advantage is **+4.69**. That means **+5.66 of value comes purely from the future** — from the outage that has not happened yet. A greedy rule cannot see that number; it only sees the 0.97.

2.1.4 B1.4 Can an explicit P be written down?

Yes, exactly. The two stochastic ingredients (demand, grid) are independent and both are small discrete distributions, so each row of P has at most $(6 + 1) \times 2 = 14$ non-zeros. We store P as a sparse $(|\mathcal{S}| \cdot |\mathcal{A}|) \times |\mathcal{S}|$ matrix — a dense $S \times A \times S$ array would be 99.9% zeros. This matters enormously for the project: because a true P exists, we can

1. run Value Iteration and get the *exact* optimum to compare against;
2. score every policy — VI's, Q-learning's, the baselines' — by **exact policy evaluation** under the true model, which removes Monte-Carlo noise from every comparison in the report;
3. deliberately hand the planner a **wrong** P and measure what the wrongness costs.

Point 3 is the reason this problem was worth building at all, and it is not hypothetical: Dhaka publishes a load-shedding schedule that is famously optimistic. A planner that believes the published schedule is a planner with a mis-specified P , and we can quantify exactly how much that belief costs it.

2.1.5 B1.5 Baselines

- **Myopic greedy** ($\gamma = 0$ value iteration) — the formally correct greedy policy, not a hand-crippled one.
- **Tuned caretaker rule** — the reactive two-threshold policy the hall actually runs, with both thresholds grid-searched.
- **Random over legal actions**, and **always idle**, as floors.
- **Certainty-equivalence VI** — build the maximum-likelihood MDP from *Q-learning's own samples* and plan on it. This is the arm that could sink the whole thesis, and we run it deliberately: if a model *learned* from n samples beats Q-learning trained on the same n samples, then “model-free wins when the model is wrong” is the wrong lesson, and the right one is narrower — a wrong *prior* model loses to a *learned* model.

Action masking is what makes these comparisons mean anything. Without it, idle and pump/grid are bit-for-bit identical in all 792 outage states, so every policy-agreement number would be measuring how two algorithms happen to break ties.

2.1.6 B1.6 Validation methodology

Exact policy evaluation (a sparse linear solve, §5.5) under the **true** model for every policy in the report, reported as **regret %** against V^* ; five independent seeds for every learned quantity, mean $\pm$ std; a check that Value Iteration's Bellman residuals decay at the rate contraction theory predicts (γ^k); a cross-check that VI's V^* agrees with the linear solve of V^{π^*} to machine precision; Monte-Carlo rollouts *only* to translate discounted return into numbers a hall warden cares about

(cost per day, dry hours per day); and a policy heatmap over (tank level $\times$ hour) that makes the pre-filling behaviour directly visible.

2.2 Problem B2 — Batch Cooking and Counter Admission in a Hall Dining Room (MDP)

2.2.1 B2.1 Informal statement

The hall dining room cooks rice and curry in **batches**, and a batch takes about 30 minutes from lighting the stove to being servable. Students do not arrive evenly: they arrive in a wall, twice a day, when classes let out. The manager decides, every ten minutes, whether to start another batch (fuel and labour, spent now, servable in half an hour) and whether to keep taking people into the queue or wave them off to the street stalls. Cook too little and the queue balks and the hall's reputation with it. Cook too much and the surplus is thrown out at closing.

2.2.2 B2.2 The MDP tuple

State $s = (R, p, q, t)$: $R \in 0 \dots 30$ ready portions in units of 5; $p \in \{0, 1, 2\}$ ten-minute slots remaining on the batch currently in the pot ($p = 0$: nothing cooking); $q \in 0 \dots 20$ students in the queue; $t \in 0 \dots 35$ ten-minute slot within a six-hour service window.

$$|\mathcal{S}| = 31 \times 3 \times 21 \times 36 = \mathbf{70,308}.$$

Actions $\{\text{wait, start a batch, close the counter}\}$; start a batch is unavailable while $p > 0$ (one pot).

Transition: arrivals per slot Poisson with a slot-dependent rate (two sharp peaks); the server clears a deterministic number of portions per slot; each waiting student balks with a fixed per-slot probability. Independent, discrete, so P is writable.

Reward: goodwill per portion served, minus a balk penalty per student who gives up, minus a waste cost per portion still sitting in the pot at close, minus the fuel/labour cost of each batch started.

Discount: the service window is a natural **episode**, so this problem is honestly finite-horizon and $\gamma = 1$ with backward induction is the right formulation. That is a real structural difference from B1, and it costs us something — see §4.8.

2.2.3 B2.3 Why it is genuinely sequential

The 30-minute cook lead time is the entire argument. A greedy rule ("the queue is long, start cooking") is *structurally* half an hour late: by the time the batch is servable the rush has balked and gone, and the food becomes waste. To serve the 13:00 wall you must light the stove at 12:30, when the dining room is empty and every greedy signal says do nothing. Lookahead is not an optimisation here; it is the only thing that works.

2.2.4 B2.4 Can an explicit P be written down?

Yes, and that is almost a problem. Arrival rates are trivially measurable — you stand at the door with a tally counter for a week — so there is no *natural* source of a badly wrong prior model. We would have to invent one, and an invented mis-specification is a much weaker experiment than B1's, where the wrong model is a real published document that real people really plan against.

2.2.5 B2.5 Baselines

The fixed cook schedule the dining hall actually runs (two batches at fixed times); a reorder-point rule (start a batch whenever $R < r$, r tuned); myopic greedy; random.

2.2.6 B2.6 Validation methodology

Same skeleton as B1 — exact evaluation under the true model, regret against the backward-induction optimum, multi-seed learning curves. The visualisation is the weak point: the state is four-dimensional with no natural pair to hold fixed, so there is no single readable policy picture, only slices.

2.3 Problem B3 — Shuttle-Bus Dispatch on the Campus Loop (MDP)

2.3.1 B3.1 Informal statement

DU runs shuttle buses on a loop through Nilkhet and Shahbagh. Buses idle at the depot; the dispatcher decides every five minutes whether to **hold** a bus (let more riders accumulate, so the trip is cheaper per rider and the crowd at the stops thins out later) or **dispatch** it now (the people waiting now stop waiting, but the bus leaves half empty and the fleet is out of position for the 17:00 rush). Holding is an investment. Dispatching is irreversible for the next forty minutes: the bus is gone.

2.3.2 B3.2 The MDP tuple

State $s = (n_{\text{depot}}, q_1..q_4, \text{ETA}_1..\text{ETA}_3, t)$: buses at the depot; queue length at each of four stops, bucketed; time-to-return of each bus en route, binned; five-minute slot over a 14-hour operating day.

Even after bucketing hard — four depot occupancies, six queue levels per stop, twelve ETA bins per bus, 168 slots —

$$|\mathcal{S}| \approx 4 \times 6^4 \times 12^3 \times 168 \approx \mathbf{1.5 \times 10^9},$$

which is not tabulable. Any tractable version requires aggregation so aggressive that the aggregated problem is a different problem.

Actions: {hold, dispatch} per idle bus. **Reward:** negative total passenger waiting time, minus fuel per trip, minus a large penalty per rider who gives up and walks.

2.3.3 B3.3 Why it is genuinely sequential

Same shape as the others: the cost of holding is paid now, the benefit is collected later, and a dispatched bus cannot be recalled. A greedy dispatcher ("send a bus whenever anyone is waiting") burns the fleet on empty trips in the quiet hour before the rush and then has nothing at the depot when 300 students appear at once.

2.3.4 B3.4 Can an explicit P be written down?

Not honestly, and this is what kills it. Travel time on the Nilkhet road is heavy-tailed and correlated with the hour, the weather and whatever is happening on the street that day. Arrivals are correlated *across* stops — a class lets out and sixty people materialise simultaneously, which is emphatically not Poisson. We could of course write *some* P down. But it would be a fiction, and the consequence is fatal for this assignment: **with no trustworthy P there is no true model to score policies against**, so there is no exact optimum, no regret axis, and no exact policy evaluation. Every comparison would degrade into Monte-Carlo rollouts with overlapping error bars. Worse, Value Iteration — half of the assignment — could not be run at all except on a model we had invented, which would make the VI-vs-Q-learning comparison a comparison between an algorithm and a fantasy.

2.3.5 B3.5 Baselines

Fixed timetable; headway-based dispatch; load-threshold dispatch ("go when the bus is 70% full or 20 minutes have passed").

2.3.6 B3.6 Validation methodology

Monte-Carlo rollouts only, with all the noise that implies. There is no exact scoring function available, which is exactly the objection above restated.

2.4 4.8 Formal selection of the decision-making project

Selected: Problem B1 — Overhead Water-Tank Pump Control, solved with Value Iteration and Q-learning.

The first two criteria are the assignment's own (Feasibility, Demonstrability). The other two are the ones this half of the assignment actually turns on: does the problem *need* lookahead, and does it let us tell a model-based algorithm apart from a model-free one in a way that means something.

Criterion	B1 · Water tank	B2 · Dining hall	B3 · Shuttle dispatch
Feasibility	★★★ $ \mathcal{S} = 1584$; sparse P ; VI converges in 2273 sweeps in 0.23 s ; a tabular Q-learner covers every legal (s, a)	★★ 70,308 states — VI still runs, but $\sim 45\times$ heavier per sweep, and tabular Q-learning needs far more samples before coverage is honest	★ $\sim 10^9$ states before aggregation; not tabulable, so neither algorithm runs as taught
Needs lookahead	★★★ myopic loses 120.62% ; even the <i>tuned</i> reactive rule loses 14.51%	★★★ the 30-min cook lead time makes any greedy rule structurally late	★★★ holding is an investment; dispatch is irreversible
Demonstrability	★★★ policy is a 2-D picture (tank level $\times$ hour): you can <i>see</i> the agent pre-fill before the evening outage band	★★ 4-D state with no natural slice; only fragments of a policy are viewable	★ a dispatch log; nothing legible
Separates VI from QL	★★★ exact P exists and a genuinely wrong prior exists in the real world (the published load-shedding schedule) — so “how wrong can your model be?” is a real experiment	★ P is writable but trivially measurable, so a wrong-prior story is contrived; and the finite horizon removes the contraction/residual experiment entirely	✗ no honest $P \rightarrow$ no VI, no exact optimum, no regret axis. The comparison cannot be made

2.4.1 Why B1 wins, in detail

B1 is the only one of the three where **both halves of the assignment are actually runnable and actually comparable**. B3 fails at the first hurdle: a problem whose transition model cannot be written down is a problem where Value Iteration is not applicable, and half the assignment evaporates — it is a model-free problem by default, which is a much less interesting thing to report than a fair fight. B2 is legitimate and we nearly took it; the lead-time argument for sequentiality is arguably even cleaner than B1's. It loses on two counts. Its state is four-dimensional with no natural 2-D slice, so the single most persuasive artifact — a picture of the optimal policy that a professor can read in three seconds — does not exist. And its finite horizon quietly deletes an entire experiment: with backward induction there are no Bellman residuals to plot, no contraction rate to check against γ^k , and no stopping-tolerance bound to justify.

B1 keeps all of it. $|\mathcal{S}| = 1584$ is small enough that Value Iteration returns the exact optimum in 0.23 s, which means every other policy in the report can be scored as a *regret against a known optimum* rather than as a number with no yardstick. The continuing, discounted formulation gives us the residual-decay check ($\gamma = 0.99$, measured empirical contraction rate **0.9900** — theory says exactly γ , and it is exactly γ). The policy is two numbers deep, so the pre-fill behaviour draws itself. And the load-shedding schedule gives us something none of the alternatives do: a

naturally occurring wrong model. We can hand the planner the optimistic published schedule and measure what optimism costs — a planner who believes outages are four times rarer than they are loses 2.66%; one who believes the grid never fails loses 21.39%; one who is only 25% too pessimistic loses 0.11%. That curve is the finding of the whole half, and only B1 could produce it.

2.4.2 4.9 The design failure that made the experiment mean something

This section is here because it is the most valuable thing we learned, and leaving it out would make the report dishonest.

The first version of B1 had an unlimited generator. The state was (L, t, g) , $|\mathcal{S}| = 528$, and the diesel action was always available at a cost of 6.0 per hour. Everything ran. Value Iteration converged, Q-learning learned, the plots were drawn, and the model-mismatch experiment — the centrepiece, the one the whole design existed to support — **measured nothing**. The regret curve was essentially flat across every wrong model we handed the planner. A planner who believed the grid never failed scored almost as well as the planner with the true model.

It took us an embarrassingly long time to see why, and the reason is completely elementary once stated. **With unlimited diesel, every mistake is recoverable one hour later at a bounded price.** Suppose the planner's optimistic model tells it not to bother pre-filling, and then the grid drops out at 18:00 with a half-empty tank. So what? It burns diesel. It pays 6.0 an hour, indefinitely, for as long as the outage lasts. The bad forecast cost it the *price difference* between grid and diesel and nothing else. There was no state the agent could get itself into that it could not buy its way out of, and so the value of knowing the future — which is the only thing a correct model buys you — was almost zero. We had built an MDP with no irreversibility in it, and then spent a week measuring how much lookahead was worth in a world where lookahead is worth nothing.

The fix was one state variable. Diesel became a finite daily ration F : two generator-hours, delivered at midnight, **not rolled over**. That is all. The state space went from 528 to 1584. And the problem changed character completely, because F is *exhaustible*: fuel burned at 07:00 on a false alarm is simply not there at 21:00. Now a wrong forecast has a consequence you cannot pay your way out of. Now the optimal policy has to reason about a resource across a whole day, and the model-mismatch experiment has something to measure — because the optimistic planner burns its ration early on outages it thinks will be short, and is holding an empty drum when the real evening outage arrives. The 21.39% figure for "the grid never fails" exists only because of that ration. In the unlimited version it would have been noise.

(The no-roll-over rule is part of the fix, not a detail: if unused fuel accumulated, the agent could hoard it indefinitely and the constraint would stop binding.)

The lesson generalises, and it is the one we would give anyone designing an MDP for a course project: **a sequential decision problem is only interesting where something cannot be undone.** Irreversibility — an exhaustible resource, a lead time, a bus that has left — is not a complication you add for realism. It is the thing that makes the future worth predicting. Take it out and the optimal policy degenerates towards the greedy one, every planning-versus-learning comparison flattens, and you are left with a working program that answers no question at all.